MACHINE LEARNING • MODULE 09

NLP Basics

Teaching Machines to Understand Language

■ NATURAL LANGUAGE PROCESSING

■ LEARNING OUTCOME

By the end of this module you will preprocess text data, extract features with TF-IDF, build text classifiers, and use HuggingFace transformers to leverage pretrained BERT models without writing complex code.

■ Skills You Will Gain

- Clean and preprocess text: tokenise, remove stopwords, lemmatise
- Create TF-IDF feature matrix for classification
- Build sentiment classifier with Logistic Regression pipeline
- Use word embeddings: Word2Vec/GloVe vs contextual (BERT)
- Apply HuggingFace transformers for zero-shot and fine-tuned NLP
- Evaluate NLP models with accuracy, F1 (macro/micro), and confusion matrix

■ **NLP powers ChatGPT, Google Search, Gmail auto-complete, Alexa, and every modern AI product. It's the fastest-growing ML subfield with the highest salaries.**

SECTION 1

Text Preprocessing Pipeline

1	**Lowercase** text.lower() — 'Apple' and 'apple' are the same word
2	**Remove Noise** URLs, @mentions, #hashtags, HTML tags, punctuation, extra whitespace
3	**Tokenise** Split text into individual tokens (words or subwords)
4	**Remove Stopwords** 'the', 'is', 'a', 'in' — common words with no discriminating information
5	**Stem or Lemmatise** Stemming: crude suffix removal ('running'->'run'). Lemmatisation: proper root form ('better'->'good')
6	**Feature Extraction** Convert cleaned tokens to numbers (TF-IDF, embeddings) for ML models

```
import re, nltk from nltk.tokenize import word_tokenize from nltk.corpus import stopwords from nltk.stem import WordNetLemmatizer nltk.download(['punkt','stopwords','wordnet'], quiet=True) stop_words = set(stopwords.words('english')) lemmatizer = WordNetLemmatizer() def preprocess(text: str) -> str: text = text.lower() text = re.sub(r'http\S+', '', text) # remove URLs text = re.sub(r'@\w+|#\w+', '', text) # remove mentions/tags text = re.sub(r'[^a-z\s]', '', text) # keep only letters tokens = word_tokenize(text) tokens = [t for t in tokens if t not in stop_words and len(t) > 2] tokens = [lemmatizer.lemmatize(t) for t in tokens] return ' '.join(tokens) df['clean'] = df['review'].apply(preprocess)
```

SECTION 2

TF-IDF Feature Extraction

TF-IDF = Term Frequency x Inverse Document Frequency

TF: How often a word appears in THIS document (normalised) IDF: log(Total docs / Docs containing the word) — penalises words that appear in many documents High TF-IDF: word appears often in this doc but rarely in other docs -> highly characteristic Low TF-IDF: word appears in most docs ('the', 'is') -> not informative

```
from sklearn.feature_extraction.text import TfidfVectorizer from sklearn.linear_model import LogisticRegression from sklearn.pipeline import Pipeline from sklearn.metrics import classification_report pipe = Pipeline([ ('tfidf', TfidfVectorizer( max_features=20000, # keep top 20K most informative tokens ngram_range=(1, 2), # unigrams + bigrams ('not good' != 'good') min_df=2, # ignore tokens in < 2 documents max_df=0.95, # ignore tokens in > 95% of documents sublinear_tf=True # log(1+tf) instead of raw tf )), ('clf', LogisticRegression(C=5, max_iter=1000)) ]) pipe.fit(X_train, y_train) print(classification_report(y_test, pipe.predict(X_test))) # Most predictive words per class feature_names = pipe['tfidf'].get_feature_names_out() coefs = pipe['clf'].coef_[0] print('Top positive:', [feature_names[i] for i in coefs.argsort()[-10:]]) print('Top negative:', [feature_names[i] for i in coefs.argsort()[:10]])
```

SECTION 3

HuggingFace Transformers

Pipeline Task	Model Example	Use Case
sentiment-analysis	distilbert-base-uncased-finetuned-sst-2	Positive/Negative sentiment
zero-shot-classification	facebook/bart-large-mnli	Classify without training data
text-generation	gpt2 / mistral	Autocomplete, creative writing
ner (named entity)	dbmdz/bert-large-cased-finetuned-conll03	Extract persons, places, orgs
summarization	facebook/bart-large-cnn	Summarise long documents
translation_en_to_fr	Helsinki-NLP/opus-mt-en-fr	Machine translation
question-answering	deepset/roberta-base-squad2	Read comprehension QA

```
from transformers import pipeline # Sentiment analysis — pretrained model sentiment = pipeline('sentiment-analysis') results = sentiment(['I loved this product!', 'Worst experience ever', 'It was okay']) for r in results: print(f"{r['label']:8s} ({r['score']:.3f})") # Zero-shot — no labelled data needed! classifier = pipeline('zero-shot-classification', model='facebook/bart-large-mnli') labels = ['sports', 'technology', 'politics', 'entertainment'] result = classifier('The new iPhone has a revolutionary AI chip', candidate_labels=labels) print(result['labels'][0], result['scores'][0]) # Named Entity Recognition ner = pipeline('ner', grouped_entities=True) entities = ner('Apple was founded by Steve Jobs in Cupertino, California') for e in entities: print(f"{e['entity_group']:5s}: {e['word']}")
```

■ PRACTICE Q & A

Q1. What is the difference between stemming and lemmatisation?

A: *Stemming: crude rule-based suffix removal ('running'->'run', 'better'->'bett'). Lemmatisation: dictionary-based — returns actual root word ('better'->'good', 'running'->'run'). Lemmatisation is slower but more accurate.*

Q2. Why do we remove stopwords in NLP preprocessing?

A: *Stopwords ('the', 'is', 'a') appear in every document and carry no discriminating information. Removing them reduces feature space and improves model focus on informative words.*

Q3. What does a high TF-IDF score indicate for a word?

A: *The word appears frequently in this specific document but rarely in the overall corpus — it's highly characteristic and informative for identifying this document's topic.*

Q4. What is n-gram range (1,2) in TF-IDF and why does it help?

A: *Include both unigrams (single words) and bigrams (two-word sequences). Bigrams capture context: 'not good' and 'not bad' have opposite meanings that unigrams miss.*

Q5. What is zero-shot classification and when is it most useful?

A: *Classify text into categories without any labelled training examples. Most useful when you have no labelled data, need to classify into new categories quickly, or are prototyping a system.*

■ NLP Cheat Sheet	
`text.lower()`	Lowercase normalisation
`re.sub(r'[^a-z\s]','',text)`	Remove punctuation
`word_tokenize(text)`	Tokenise into words
`stopwords.words('english')`	English stopwords list
`lemmatizer.lemmatize(word)`	Get canonical word form
`TfidfVectorizer(max_features=20000)`	TF-IDF feature matrix
`ngram_range=(1,2)`	Unigrams + bigrams

`pipeline('sentiment-analysis')`	HuggingFace sentiment
`pipeline('zero-shot-classification')`	Classify without labels
`pipeline('ner', grouped_entities=True)`	Named entity recognition
`pipeline('summarization')`	Document summarisation
`classification_report(y, yp)`	Full P/R/F1 per class